

台北都更專案技術棧與資料清洗能力

把混亂的城市開放資料清成可查詢、可定位、可分析的資料產品

江慶于414580662

2026-05-09

Contents

0.1 交付範圍	1
0.2 技術棧	1
0.3 後端分層架構	2
0.4 R 分析層	3
0.5 原始資料為什麼很髒	3
0.6 資料清洗能力	3
0.7 API 能力	4
0.8 為什麼這是強資料清洗能力	5
0.9 技術亮點摘要	5
0.10 結語	5

0.1 交付範圍

本文件介紹本專案的整體技術棧與資料工程能力。專案由三個部分組成：Python FastAPI 後端服務、地址解析與地理編碼的清洗腳本、以及 R Markdown 行政區層級分析報告。目標是把台北市都市更新、地震危險建築物、海砂屋等髒資料清成可查詢、可定位、可分析的資料產品。

這個專案的難點不是「把 JSON 讀出來」而已。原始資料包含大量地址文字、跨區描述、括號備註、門牌範圍、巢狀空間座標與缺漏欄位。如果不先清洗，後續 API、地圖、統計分析都會出現錯誤或偏差。

0.2 技術棧

以下表格整理本專案實際使用的技術，涵蓋後端服務、資料清洗腳本與 R Markdown 分析三個層次。

Table 1: 專案技術棧

類別	技術	角色
API 框架	FastAPI	建立 REST API、Swagger 文件與資料查詢端點
執行環境	Python 3.12、uvicorn	提供可重現的後端執行環境
套件管理	uv、uv.lock	鎖定依賴版本，降低不同機器環境差異
資料驗證	Pydantic Models	把外部髒資料轉成有 schema 的穩定資料結構
HTTP 資料擷取	httpx / AsyncClient	擷取台北市開放資料與外部地理編碼服務
空間資料處理	GeoJSON、Shapely、STRtree、Haversine	處理都市更新多邊形、附近搜尋與距離計算
地理編碼	ArcGIS Geocoding API	將只有地址的資料補上經緯度
資料分析與報告	R Markdown、httr2、dplyr、tidyr、readr	直接串接本機 API 產生分析表格與彙整資料
地圖視覺化	ggplot2、ragg、coord_quickmap、geoBoundaries TWN ADM2	繪製 12 區行政區界 + 危建建物分布地圖 (圖 5)
測試與品質部署	pytest、ruff Dockerfile	驗證 model 行為與維持程式碼品質 讓服務可被容器化部署

Table 2: 後端專案規模指標

指標	數值
後端 Python 檔案	28
API Handler 模組	6
資料模型模組	5
GET API 端點	17

0.3 後端分層架構

後端採用分層設計，不把資料讀取、清洗、查詢與 API 回傳混在同一層。這讓資料源很髒時，仍能把外部混亂擋在 service / cleaning layer，對外提供穩定的 API 給上層分析使用。

Table 3: 後端分層架構

層級	對應位置	責任
Router	backend/app/router.py	集中註冊 API endpoint，提供 Swagger 文件與查詢參數
Handler	backend/app/handler	封裝各資料主題的 API 操作，例如危建、海砂屋、都市更新
Service Model	backend/app/services backend/app/models	處理資料讀取、附近搜尋、空間計算與彙整邏輯 用 Pydantic 定義輸入輸出 schema，避免資料格式漂移
Cleaning Scripts	backend/data/*_cleaning.py	把原始髒資料轉成後端與分析端可用的乾淨資料
Public Data	backend/public、backend/ data/*.json	保存清洗後或預處理後的 JSON / GeoJSON 資料

0.4 R 分析層

清洗後的 API 資料由 R Markdown 報告以 `httr2` 串接讀入，再以 `dplyr` 與 `tidyr` 完成行政區層級的合併、缺漏值處理與危建總數計算，最後用 `ggplot2` 產生 5 張視覺化、`knitr::kable` 輸出表格、並以 `readr::write_csv` 將乾淨的合併資料輸出為 CSV 供後續使用。

地圖視覺化（圖 5）疊加三層資料：底圖為 `geoBoundaries` TWN ADM2 行政區界（`/api/geojson/taipei-districts`），中層為清洗後的逐棟危建點位（`/api/geojson/earthquake-buildings`、`/api/geojson/chloride-ionized-concrete`），最上層為各區的中文標籤（位置由 `Shapely` `representative_point` 預先算入 `GeoJSON` `properties`，確保標籤落在多邊形內部）。

這層的職責是分析與呈現：依賴後端 API 提供穩定的資料 `schema`，不重複處理地址文字或空間座標，讓資料工程與統計分析在不同層各司其職。

0.5 原始資料為什麼很髒

這批資料的困難點在於「它看起來像表格，但很多關鍵資訊其實藏在文字裡」。地址欄位同時承載門牌、範圍、樓層、備註與多棟建物；都市更新資料則包含跨區案件與複雜多邊形座標。如果直接拿來統計或上圖，結果很容易失真。

Table 4: 髒資料問題與清洗策略

髒資料問題	典型例子	清洗作法
一格地址包含多棟建築	同一筆 <code>building_location</code> 描述多棟建物	拆成多筆可定位資料
門牌是範圍而非單點	16至22號、17至33之1號	用規則展開連續門牌
頓號枚舉多個門牌	2、4、6號	用 <code>regex</code> 展開枚舉門牌
括號內混入行政備註	其中某棟撤銷列管等文字	移除不影響定位的括號備註
樓層文字干擾定位	1至5樓、地下一樓等資訊	移除樓層資訊後再做地址解析
台／臺用字不一致	台北市與臺北市混用	統一成同一種字形再 <code>join / group by</code>
都市更新案件跨行政區	士林區、北投區	拆開後重新彙總到各行政區
缺少經緯度	只有地址，沒有 <code>longitude / latitude</code>	串接 <code>ArcGIS Geocoding</code> 補座標
座標可能是 <code>Polygon</code> 或 <code>MultiPolygon</code>	都市更新範圍不是點，而是一片基地範圍	用 <code>Shapely</code> 建成幾何物件並支援空間查詢

0.6 資料清洗能力

0.6.1 地址解析：從文字描述變成可定位門牌

地震危險建築物資料最麻煩的地方，是 `building_location` 不是乾淨地址，而是一段半結構化文字。清洗腳本會做以下處理：

- 移除括號備註，避免「撤銷列管」等行政文字干擾定位。
- 移除樓層描述，因為 `geocoding` 需要的是地址，不是樓層。
- 以分號切段，處理同一筆資料中多個地址段落。
- 記錄上一個道路名稱，讓後續只寫巷弄門牌的片段也能補齊路名。
- 展開 `表示` 的門牌範圍。

- 展開 表示的多個門牌。

這讓原本一筆模糊文字可以被拆成多個可定位地址，大幅提高資料可用性。

0.6.2 地理編碼：補上原始資料沒有的座標

很多城市開放資料只提供地址，沒有經緯度。後端透過 ArcGIS Geocoding API，把「縣市 + 地址」轉成 WGS84 經緯度。

這一步的價值是把資料從「只能讀文字」提升為「可以做空間分析」：

- 可以在地圖上定位。
- 可以與都市更新範圍疊合。
- 可以做附近搜尋。
- 可以輸出給 R Markdown 做行政區分析與視覺化。

0.6.3 空間查詢：都市更新不是點，是面

都市更新資料常是基地範圍，不是單一座標點。後端使用 Shapely 將座標建立成 Polygon 或 MultiPolygon，再用 STRtree 建立空間索引。

這代表系統可以回答更接近真實城市問題的查詢：

- 使用者座標是否落在某都市更新範圍內。
- 使用者附近一定半徑內有哪些都市更新案件。
- 都市更新案件距離指定座標多遠。

Table 5: 資料清洗與服務化流程

階段	輸入	輸出
擷取	台北市開放資料 / 本地 JSON	原始資料
驗證	原始 JSON	符合 Pydantic schema 的資料
文字清理	地址與備註混雜文字	較乾淨的地址字串
地址展開	範圍、枚舉、多地址文字	多筆可定位門牌
座標補齊	地址	longitude / latitude
空間化	座標陣列與 GeoJSON	Point / Polygon / MultiPolygon
API 化	清洗後資料	FastAPI endpoint
分析化	本機 API	R Markdown 表格與圖表

0.7 API 能力

清洗後的資料透過 FastAPI 包裝成穩定服務，而不是散落在不同腳本裡。

Table 6: 後端 API 能力

API類型	能力
地震危險建築物	回傳列管危建清單與單筆資料

API類型	能力
海砂屋	回傳高氯離子混凝土建築資料
都市更新	依行政區整理都市更新案件與案件數
危建建物 GeoJSON	輸出可被地圖或空間分析使用的 FeatureCollection
行政區界 GeoJSON	提供 12 個行政區的多邊形與標籤位置，作為地圖底圖
附近搜尋	依座標與半徑搜尋附近都市更新案件
健康檢查	確認服務狀態與版本
分析報告串接	R Markdown 可直接呼叫 localhost API 產生分析報告與地圖

0.8 為什麼這是強資料清洗能力

本專案的資料清洗不是表面上的欄位改名，而是把原本難以計算、難以定位、難以合併的城市資料，轉成後端 API、空間查詢與統計分析都能使用的穩定資料層。

具體來說，清洗能力體現在四件事：

1. 可定位：原始地址文字經過解析與 geocoding 後，可以變成經緯度。
2. 可合併：行政區名稱、跨區案件與台臺用字被標準化後，可以做 join 與 group by。
3. 可查詢：都市更新多邊形經過 Shapely 處理後，可以做附近搜尋與距離計算。
4. 可分析：R Markdown 可以直接呼叫 API，產出危建與都更追趕率分析。

0.9 技術亮點摘要

Table 7: 後端與資料清洗技術亮點

亮點	專案價值
FastAPI 服務化	把清洗成果提供成穩定 API，而不是一次性檔案
Pydantic schema 驗證	避免髒資料欄位漂移造成前後端與分析失敗
半結構化地址解析	把難處理的地址文字拆成可定位資料
ArcGIS 地理編碼	補上原始資料缺少的經緯度
Shapely 空間計算	讓都市更新範圍可以被附近搜尋與距離計算使用
行政區界整合與地圖視覺化	整合 geoBoundaries 行政區界與危建點位，呈現空間叢集
R Markdown 分析串接	同一份後端資料可直接產出分析報告與視覺化

0.10 結語

這個專案真正的價值，是把非常髒的都市開放資料清成可重複使用的資料產品。後端負責資料品質控管、地址解析、座標補齊與空間查詢；R Markdown 報告承接後端輸出，完成行政區層級的合併、視覺化與追趕率分析。

我們把原本混亂、難定位、難合併、難分析的城市資料，整理成可驗證、可查詢、可定位、可分析的資料產品，並透過 R Markdown 完成行政區層級的分析與報告。